

Cache 与存储层次

怎样维护一个正确的高速副本，并把命中/缺失变成可靠的性能模型

408 · 计算机组成原理 Topic CO-06

母问题：小而快的副本怎样既保证“取对数据”，又真正降低平均访问成本？

Cache 的知识不是若干并列名词，而是一条生成链：

Locality → Block Transfer → Placement
→ Identity / State → Hit or Miss
→ Replacement / Write → Hierarchy Cost

这些箭头表示**解释依赖**：局部性说明为什么值得保存副本；块搬运决定副本粒度；映射给出候选位置；tag/valid 等状态证明身份；只有状态轨迹确定后，命中率与 AMAT 才有可靠输入。

本册负责什么

- **Own**: block/line、set/way、tag/index/offset、hit/miss、3C、replacement、write-through/write-back、write-allocate/no-write-allocate、Cache 物理位数、local/global miss rate 与 AMAT。
- **Use**: CO-05 提供下一级主存/总线服务时间；CO-07 提供翻译后的地址与权限；CO-04 决定 miss 是否真的形成流水线 stall。
- **Stop Boundary**: 页表、TLB page walk 与 fault 修复不在本册重新定义；OS Page Cache 也不是 CPU Cache 的同一个状态机。

目录

1	为什么需要层次：单一介质同时满足不了速度、容量与成本	3
2	对象与状态：先知道 Cache 到底保存什么	3
2.1	地址字段：定位候选与定位块内字节	3
2.2	地址位预算是不变量：几何参数变化时先看谁把位让给谁	3
2.3	数据容量与物理总位数	4
3	三类映射：先找候选，再证明身份	4
4	Hit/Miss 生命周期：Cache 是状态机，不是命中率公式	5
4.1	Read hit	5
4.2	Read miss	5

5 写策略：传播时机与写 Miss 行为是两条独立轴	6
6 替换、3C 与反事实基线	7
7 性能语言：先定义路径，再算 AMAT	7
7.1 先分清时间和速率	7
7.2 Hit Time、Miss Total Time 与 Miss Penalty	7
7.3 AMAT 是路径时间的期望	7
7.4 多级 Cache：只有走到下一层才支付下一层成本	8
8 地址翻译接口：翻译层次、数据层次和虚存慢路径不是一条链	8
9 贯穿母例：两个块反复映射到同一组	9
10 概念边界：相似词必须改变做题行为	9
11 做题控制协议：从请求到成本只走一次	9
12 本册与其他 Owner 的接口	10

1 为什么需要层次：单一介质同时满足不了速度、容量与成本

存储系统同时追求更低延迟、更大容量和更低每位成本。寄存器、SRAM Cache、主存（408 中通常由 DRAM 实现）与后备存储在这些轴上取舍不同，因此系统利用局部性把近期更可能使用的信息保留在更近的层。

时间局部性表示近期访问对象可能再次被访问；**空间局部性**表示附近地址可能很快被访问。局部性只回答“为什么缓存可能有收益”，不证明某次访问一定命中。实际命中率还取决于块大小、容量、映射、相联度、替换策略和访问序列。

寄存器位于存储阶梯最上方，但由指令显式命名，不经过 Cache 式 placement/tag/replacement，因此不能机械当作又一级 Cache。

2 对象与状态：先知道 Cache 到底保存什么

核心对象

- **Block / Cache line**: Cache 与下一级之间搬运的数据粒度；line 是 Cache 中承载一个 block 副本的位置。
- **Set / Way**: set 是一次查找的候选集合；way 是组内的一条候选 line。
- **Tag**: 证明候选 line 对应哪个下一级 block 的身份字段。
- **Valid**: 说明当前 line 是否包含可用副本；tag 相同但 valid=0 仍不能命中。
- **Dirty**: write-back 下说明 Cache 副本比下一级更新，逐出前可能需要写回。
- **Replacement state**: 组内 victim 选择所需的历史/近似状态；具体编码依实现与题设。
- **Pending / ready**: 综合时序题中，miss 请求是否在途、返回数据何时可用；是否需要这类状态由题设决定。

2.1 地址字段：定位候选与定位块内字节

设主存地址宽度为 A 位，Cache 数据容量为 C 字节，块大小为 $B = 2^b$ 字节，相联度为 E ，则组数

$$S = \frac{C}{BE}, \quad s = \log_2 S,$$

字节编址时

$$offset = b, \quad index = s, \quad tag = A - b - s.$$

若题目按字编址，必须先把“块大小”换成每块包含多少个**编址单元**，不能直接沿用字节偏移位数。

地址字段属于**请求**；Cache line 中保存的是 data 与 metadata。index/offset 并不是每一行都重复存储的字段。

2.2 地址位预算是不变量：几何参数变化时先看谁把位让给谁

对字节编址、2 的幂几何，地址宽度固定为 A 时始终满足

$$A = tag + index + offset.$$

这条等式比记某一道“块变大以后 Tag 怎样变”的结论更稳定。结合

$$S = \frac{C}{BE}$$

可以直接生成常见微扰：

- 固定 Cache 数据容量 C 与相联度 E ，块大小 B 加倍： $offset + 1$ ，组数减半所以 $index - 1$ ，二者抵消，因此 tag 不变；
- 固定块大小和总 line 数，相联度 E 加倍：组数减半，所以 $index - 1$ ； $offset$ 不变，因此 $tag + 1$ ；
- 固定地址宽度与组数，块大小加倍： $offset + 1$ 而 $index$ 不变，因此 $tag - 1$ 。

所以“相联度提高”“块变大”都不能脱离**哪些总量保持不变**直接推出 Tag 变化。

直接映射还有一个可作独立校验的数量关系。若主存与 Cache 使用相同块大小，主存块数与 Cache line 数之比为 2^{tag} ，则

$$tag = \log_2 \left(\frac{N_{memory\ blocks}}{N_{cache\ lines}} \right).$$

在主存容量与 Cache 数据容量均按相同编址粒度计量时，这个比值也等于两者容量之比。它只是直接映射、规则 2 的幂组织下的派生检查，不替代地址三分。

2.3 数据容量与物理总位数

若共有 N_{lines} 条 line，则数据区位数为

$$C_{data} = N_{lines} \times B \times 8.$$

物理实现还可能包含 tag、valid、dirty、replacement、ECC 等：

$$C_{physical} = \sum_{line} (data + tag + valid + dirty + metadata).$$

题目说“Cache 容量”时先确认是 data-only，还是要求总物理位数。

3 三类映射：先找候选，再证明身份

组织	候选范围	主要收益	主要代价
直接映射	每组 1 条 line	查找和选择简单	冲突风险高
E 路组相联	同组 E 条 line	冲突与硬件成本折中	多 tag 比较、victim 选择
全相联	全部 line	映射冲突最少	比较器、功耗和替换成本最高

直接映射常写

$$set = block\ number \bmod S.$$

组相联先由 index 定位一组，再并行/逻辑上比较组内候选 tag；全相联不需要 set index。相联度增大通常能减少 conflict miss，但也可能增加比较器、功耗和 hit path 延迟，不能只说“越大越好”。

Cache 三种映射：改变的是“候选 line 有多少”，身份仍靠 Valid + Tag 证明

同一个主存 block b 在三种组织中寻找副本；图只比较 placement/candidate 范围，不比较替换策略或写策略。

主存 Block 1 2 3 4 b 6 7

直接映射

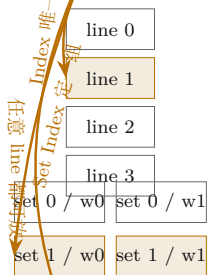
[Tag | Index | Offset]

2 路组相联

[Tag | Set Index | Offset]

全相联

[Tag | Offset] (没有 Set)



候选数 = 1: 只比较 1 个 Tag。硬件简单, 但映射到同一 line 的不同 block 不能共存, 冲突风险最高。

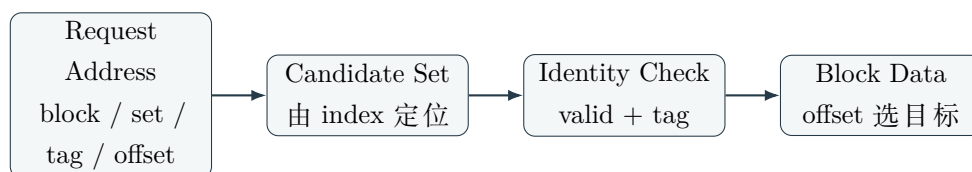
候选数 = $E = 2$: 组内两个 Tag 都要参与匹配; 同组的两个活跃 block 可以共存, 所以“same set”本身不等于 conflict miss。

候选数 = 全部 line: 映射冲突最少, 但需要更广的 Tag 比较与 victim 选择, 硬件/功耗/时延代价更高。

Index/Set 只缩小候选集合; 是否 Hit 最终仍要 Valid=1 且 Tag 匹配。

地址字段属于请求; Cache line 保存 Data + Tag + Valid/Dirty/Replacement 等 metadata, 不能把 Index/Offset 机械算成每行都存一份。

同一个主存 block 在三种组织中只改变“候选 line 范围”; 命中仍必须由 Valid 与 Tag 共同证明。图中的地址字段属于请求, 不表示每条 Cache line 都保存 Index/Offset。



上图箭头表示**查找依赖**, 不是要求实现必须四拍串行; 现代 Cache 可以并行读取 tag/data, 但逻辑上仍要同时满足候选位置、身份有效和块内定位三个条件。

4 Hit/Miss 生命周期: Cache 是状态机, 不是命中率公式

4.1 Read hit

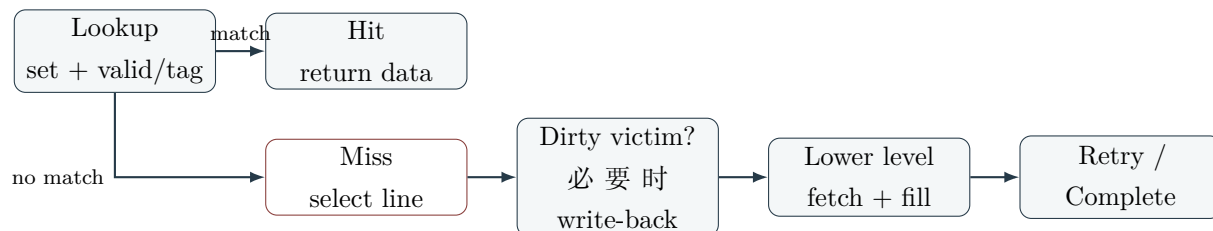
定位候选组后, 存在一条 line 同时满足 valid=1 且 tag 匹配, 再用 offset 选出目标字节/字。命中只证明“本层已有可用目标副本”, 不表示下一级没有副本, 也不等价于地址翻译成功。

4.2 Read miss

最小生命周期为

detect miss → choose invalid/victim → write back if dirty
→ fetch block → fill data/tag/state → retry or complete.

如果存在 invalid line, 通常可直接使用; 若必须逐出 victim, write-back Cache 还要检查 dirty。非阻塞 Cache、critical-word-first、MSHR 等只有题设明确时才加入; 它们改变时序和重叠, 不改变副本身份要求。



这张图表示状态转移与慢路径责任；具体实现可重叠某些阶段。

5 写策略：传播时机与写 Miss 行为是两条独立轴

命中时：

- **write-through**：同时更新下一级，状态关系直接，但写流量较大；
- **write-back**：先只更新 Cache 并置 dirty，逐出时才写回，降低平均写流量但增加状态与逐出成本。

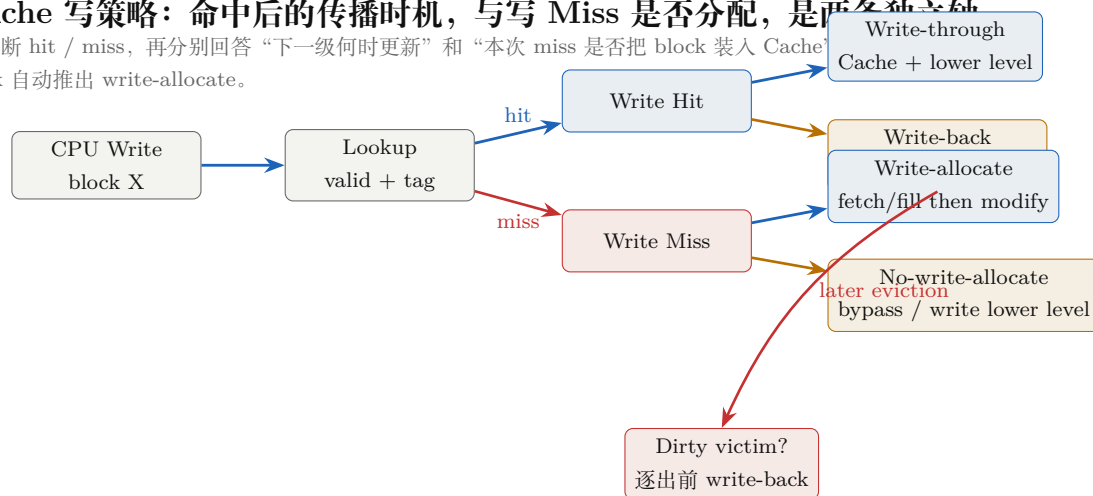
写 miss 时：

- **write-allocate**：先把目标 block 取入 Cache，再在本层修改；
- **no-write-allocate**：不为这次写分配 Cache line，写请求直接向下传播。

常见搭配不是逻辑必然。write-back 不自动推出 write-allocate，做题必须分别判断。

Cache 写策略：命中后的传播时机，与写 Miss 是否分配，是两条独立轴

先判断 hit / miss，再分别回答“下一级何时更新”和“本次 miss 是否把 block 装入 Cache”
back 自动推出 write-allocate。



Dirty 的语义：只在 write-back 路径中表示“本层副本比下一级新”。它不是“这一行曾经被访问过”，也不是 write-through 的必需元数据。Dirty victim 被逐出时才需要把较新的整块状态传播到下一级。

写缓冲可以隐藏一部分前端等待，但不会创造下一级长期带宽；持续到达率超过排空率时仍会反压。

写缓冲的边界

write-through 若要求 CPU 等每次下一级写完成，会直接暴露慢层延迟。write buffer 把待写请求先排入有限队列，在缓冲未滿时允许前端继续；write-back 的 dirty victim 也可能进入 write-back buffer。二者都用队列吸收短时突发，但没有创造下一级长期带宽：若到达率持续高于排空率，最终仍会反压前端。

6 替换、3C 与反事实基线

直接映射没有 victim 选择；组相联/全相联可使用 FIFO、Random、LRU 或近似策略。精确 LRU 的元数据与更新逻辑随相联度和编码方案变化，不能统一断言“每行只需要 $\log_2 E$ 位”。

3C 分类必须绑定反事实基线：

- **Compulsory miss**：该 block 第一次进入当前考察层；
- **Capacity miss**：即使换成同容量全相联基线，工作集仍装不下导致的 miss；
- **Conflict miss**：有限映射相对同容量全相联基线额外产生的 miss。

块大小、容量和访问序列改变后，分类结果也可能改变。

在同一引用串、全相联且采用栈算法（如经典 LRU）的模型中，增大容量不会增加 miss 数；Belady anomaly 是非栈策略可能出现的性质。这个结论不能越过“全相联/同一引用串/策略”条件直接套到任意 Cache 组织。

7 性能语言：先定义路径，再算 AMAT

7.1 先分清时间和速率

Latency 是一般的请求到结果可用等待；**Access Time** 是某存储器件/接口完成规定访问的时间；**Cycle Time** 强调同一资源下一次合法启动操作的间隔。**Throughput** 是单位时间完成的请求/事务数，**Bandwidth** 是单位时间传输的有效数据量。多体交叉和 burst 常主要提高吞吐量/带宽，并不保证单次访问 latency 同比例下降。

7.2 Hit Time、Miss Total Time 与 Miss Penalty

若当前层 hit 路径总时间为 T_{hit} ，miss 从原请求开始到最终完成的总时间为 $T_{miss,total}$ ，本册规范化定义

$$MP = T_{miss,total} - T_{hit},$$

即 miss penalty 是**相对 hit 的额外成本**。若题面“缺失损失 200 cycles”未说明是额外代价还是完整 miss path，不要擅自把它记作 MP ；先保留 $T_{miss,total}$ 再由题意判断。

7.3 AMAT 是路径时间的期望

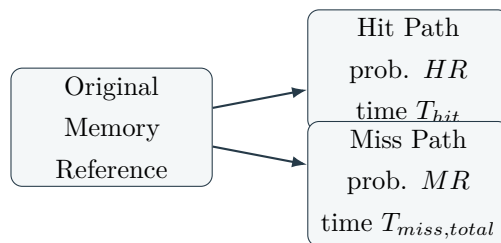
若 hit rate 为 HR 、miss rate 为 $MR = 1 - HR$ ，则定义式为

$$AMAT = HR T_{hit} + MR T_{miss,total}.$$

在上面的 MP 定义下才可化简为

$$AMAT = T_{hit} + MR \times MP.$$

它不是第二条独立公式，而是条件期望的代数化简。



图中的分支是**互斥完成路径**；真实硬件是否并行、重叠，要先画事件时间线再决定 T_{hit} 和 $T_{miss,total}$ 包含什么。

7.4 多级 Cache：只有走到下一层才支付下一层成本

若 L1 hit time 为 t_1 、local miss rate 为 m_1 ；L2 被访问后的 hit time 为 t_2 、local miss rate 为 m_2 ；L2 miss 后主存服务时间为 t_M ，并暂不考虑额外写回/重叠，则

$$AMAT_{L1} = t_1 + m_1(t_2 + m_2 t_M) = t_1 + m_1 t_2 + m_1 m_2 t_M.$$

L2 的 **local miss rate** 以实际到达 L2 的请求为分母；**global miss rate** 以 CPU 原始 references 为分母。在“每个 L1 miss 恰好访问一次 L2、无 bypass/prefetch 额外请求”的简单模型中，L2 global miss rate 为 $m_1 m_2$ 。

下一级 AMAT、脏 victim 写回、总线仲裁、refill 与 retry 都可能进入上一级 miss path。若题设允许 non-blocking Cache、write buffer、prefetch 或流水线隐藏部分等待，某些时间可能相加、取最大，或根本不形成 CPU stall。**AMAT 是每个 memory reference 的接口平均成本，不等于整个程序的 CPU time。**

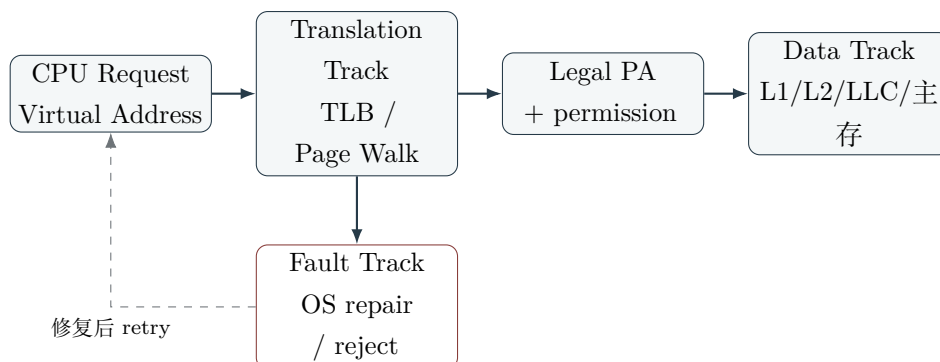
8 地址翻译接口：翻译层次、数据层次和虚存慢路径不是一条链

PIPT、VIPT、VIVT 是 Cache 与地址翻译的接口组织：PIPT 等翻译得到 PA 后查 Cache；VIPT 可用页内偏移中的 index 与 TLB 并行，再用翻译后的物理 tag 确认身份；VIVT 则需要处理 synonym/homonym 等额外一致性问题。

若 page offset 为 p 位、block offset 为 b 位，VIPT 在经典约束下可用于 set index 的位数至多为 $p - b$ ，等价写成

$$sets \times block\ size \leq page\ size.$$

这个限制来自“index 必须完全落在翻译不改变的 page offset 中”，不是固定容量口诀。



上图箭头表示**语义依赖与控制转移**。不能把它压成无条件的‘TLB → L1 → L2 → 主存 → 磁盘’：TLB 缓存翻译，Cache 缓存数据块，页面不驻留时才进入 OS 的 fault 慢路径。

9 贯穿母例：两个块反复映射到同一组

设直接映射 Cache 有 S 组、块大小 B ，两个主存块 x 与 y 满足

$$x \bmod S = y \bmod S, \quad x \neq y.$$

它们交替访问时都会定位同一条候选 line，但 tag 不同，因此可能反复互相驱逐。这个例子同时验证：

1. index 只负责定位候选，不证明身份；

2. tag/valid 才决定 hit；

3. 直接映射冲突来自 placement 约束，而不是 Cache 其余空间真的已满；

4. 提高相联度可允许两块在同组共存，但会增加比较和替换成本；

5. 命中率变化最终还要连同 hit time 与 miss penalty 才能判断性能收益。

10 概念边界：相似词必须改变做题行为

边界	为什么容易混淆	真正判据 / 题面信号	混淆后果
地址字段 ≠ line 内容	都出现 tag/index/offset	请求字段用于定位；line 只保存 data+metadata	错算物理容量
Hit rate ≠ Performance	命中高常常更快	还需 hit time、penalty、写流量与重叠	只凭命中率判快慢
Miss penalty ≠ Miss total time	都会被叫“缺失时间”	前者是额外成本；后者含当前层查找	重复或漏算 hit time
Local MR ≠ Global MR	都写 miss rate	分母是本层 accesses 还是 CPU refs	多乘/少乘上层 miss rate
Write-back ≠ Write-allocate	常作为搭配出现	一个管命中后的传播，一个管写 miss 是否取块	从一轴错误推出另一轴
Conflict ≠ Capacity	都会逐出 line	同容量全相联反事实基线是否仍 miss	选错优化方向
Latency ≠ Bandwidth	都描述“快”	单请求等待 vs 单位时间数据量	用带宽推出随机访问延迟
Cache miss ≠ Page Fault	都进入慢路径	数据块副本缺失 vs 翻译/驻留/权限 fault	把硬件 miss 写成 OS 调页

11 做题控制协议：从请求到成本只走一次

1. **Reference:** 先确认题目统计哪些 instruction/data references，一条源语句可能产生多次访问；

- 2. **Granularity**: 统一编址单位、block size、Cache data size、set/way;
- 3. **Address**: 求 block/set/tag/offset, 不把地址字段写进 line metadata;
- 4. **State**: 逐次维护 valid/tag/dirty/replacement;
- 5. **Lifecycle**: miss 时写 victim、write-back、lower-level fetch、fill、retry; 写 miss 再判断 allocate;
- 6. **Count**: 先数 hit、miss、write-back、lower-level transactions;
- 7. **Cost**: 声明 hit time、miss total / extra penalty、local/global miss rate 和重叠;
- 8. **Verify**: 检查访问总数、块数上界、概率范围、地址位数和独立时间口径。

陌生题第一动作

先问两句话：“当前请求对应哪个 block/set/tag/offset ?” “当前 line 的身份与状态是否足以证明这是目标副本 ?” 只有这两问闭合后，才进入命中率和 AMAT。

12 本册与其他 Owner 的接口

相邻 Owner	本册使用什么	什么时候停止本册
CO-05 主存与存储硬件	下一级 latency、burst、bank、bandwidth	开始推 DRAM/总线事务内部细节
CO-07 地址翻译	合法 PA、permission、PIPT/VIPT 条件	开始推 VPN/PFN、page walk、TLB 状态
CO-04 流水线	miss stall / overlap 边界	开始问哪条指令何时 ready/commit
OS-04 / X-B02	Page Fault 后的 repair/retry 结果	开始分配 frame、page-in、COW、replacement

一页压缩

Cache 先解决**正确副本**，再讨论**平均性能**：

Reference → Block/Set/Tag/Offset → Valid Identity Check
→ Hit or Miss State Transition → Event Count → Path Cost / AMAT

忘掉具体公式时，从“候选位置 + 身份 + 状态 + 慢路径 + 路径期望”即可重新生成主要结论。